بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

37 - Operation of Datapath

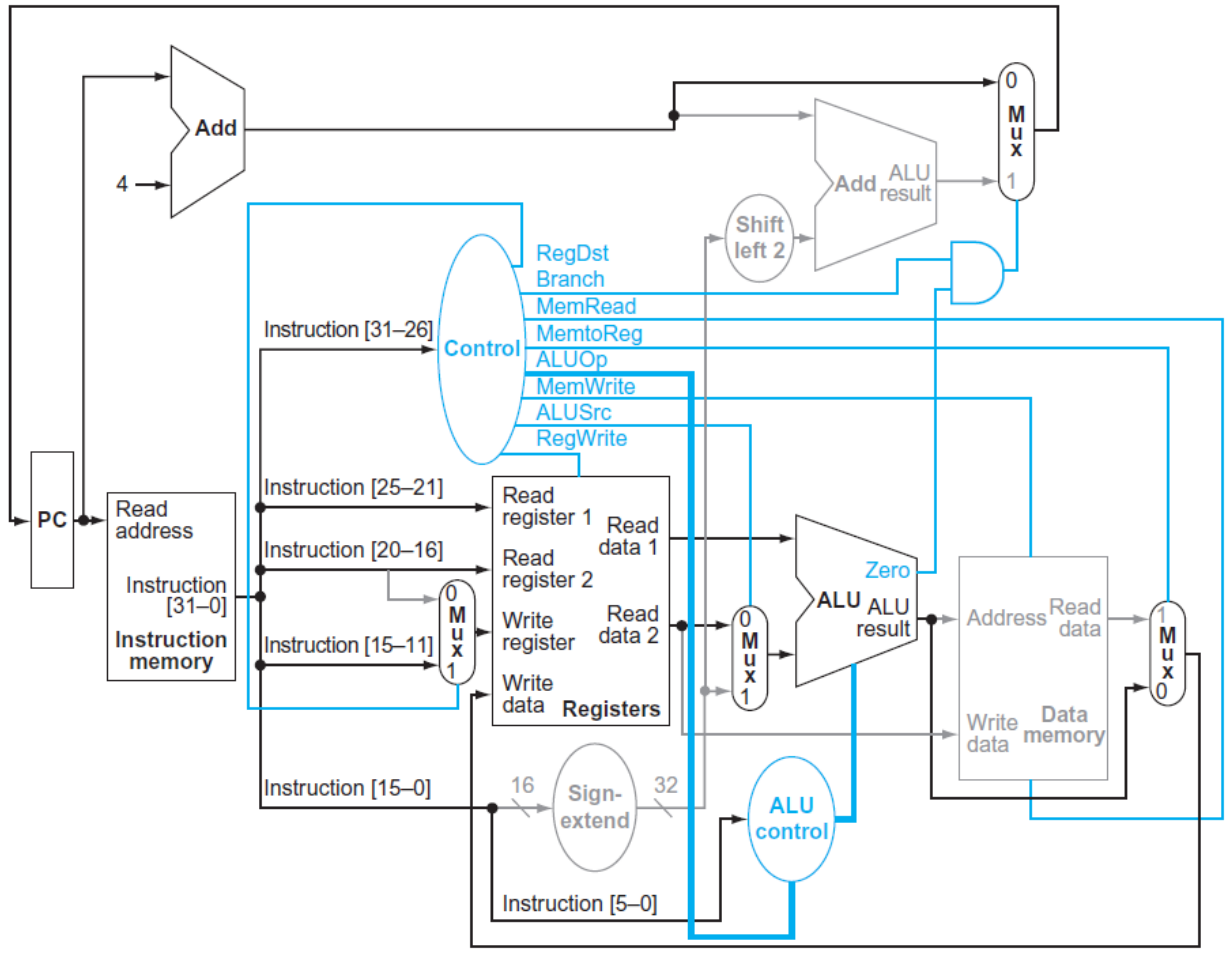
Introduction

- Look at how each instruction uses datapath
- Show flow of 3 different instruction classes through datapath
- Asserted control signals and active datapath elements are highlighted in each of these
- A multiplexor whose control is 0 has a definite action, even if its control line is not highlighted
- Multiple-bit control signals are highlighted if any constituent signal is asserted

Datapath Execution of R-type Instruction

- Figure shows operation of datapath for an R-type instruction, such as add $t1, $t2, $t3
- Although everything occurs in one clock cycle, we can think of four steps to execute instruction; these steps are ordered by flow of information

Datapath Execution of R-type Instruction



Datapath Execution of R-type Instruction

1. Instruction is fetched, and PC is incremented
2. Two registers, $t2 and $t3, are read from register file; main control unit computes setting of control lines during this step
3. ALU operates on data read from register file, using function code (bits 5:0, which is *funct* field, of instruction) to generate ALU function
4. Result from ALU is written into register file using bits 15:11 of instruction to select destination register ($t1)

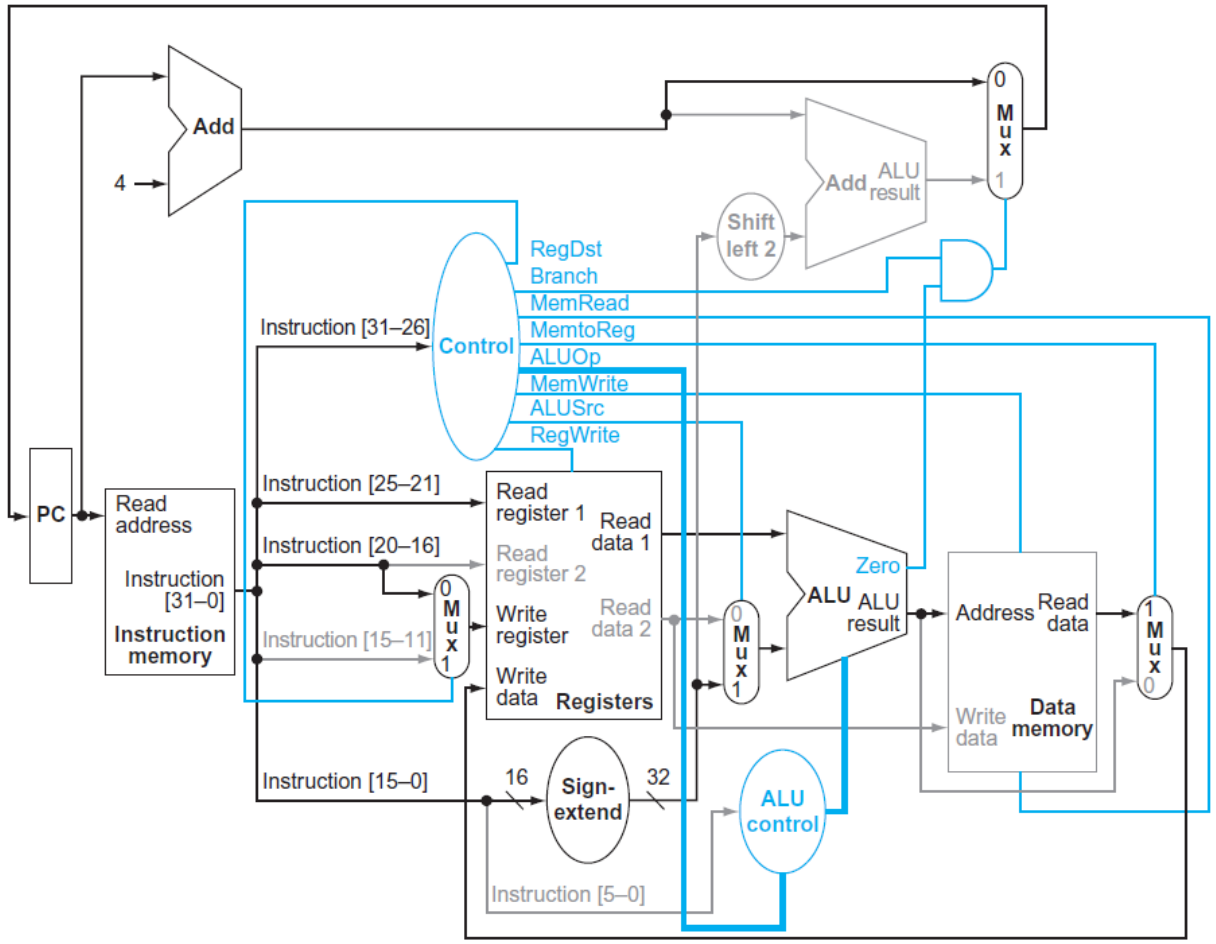
Execution of Load Word

- Such as lw $t1, offset($t2)
- Figure shows active functional units and asserted control lines for a load
- We can think of a load instruction as operating in 5 steps

Execution of Load Word

1. Instruction is fetched from instruction memory, PC is incremented
2. Register ($t2) value is read from register file
3. ALU computes sum of value read from register file and sign-extended, lower 16 bits of the instruction (offset)
4. Sum from ALU is used as address for the data memory
5. Data from memory unit is written into register file; register destination is given by bits 20:16 of instruction ($t1)

Execution of Load Word



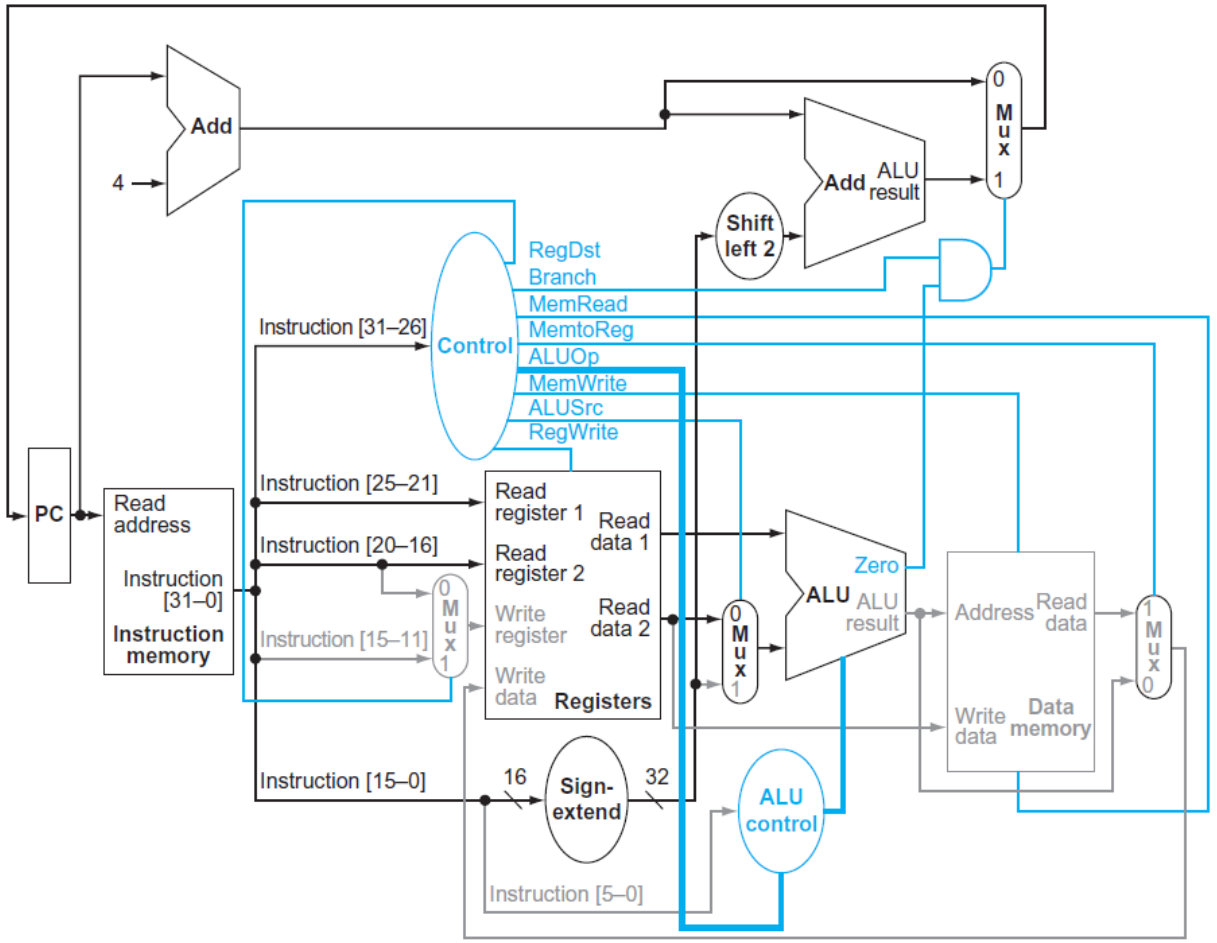
Execution of Branch-on-equal

- Such as beq $t1, $t2, offset
- It operates much like an R-format instruction, but ALU output is used to determine whether PC is written with PC + 4 or the branch target address
- Four steps in execution

Execution of Branch-on-equal

1. Instruction is fetched from instruction memory, PC is incremented
2. Two registers, $t1 and $t2, are read from register file
3. ALU performs subtract on data values read from register file; value of PC + 4 is added to sign-extended, lower 16 bits of instruction (offset) shifted left by two; result is the branch target address
4. Zero result from ALU is used to decide which adder result to store into PC

Execution of Branch-on-equal



Finalizing Control

- Now that we have seen how the instructions operate in steps, let's continue with control implementation
- Control function can be precisely defined using the setting of control lines
  - Input is 6-bit opcode field, Op [5:0]
  - Outputs are control lines

Setting Control Signals by Opcode

Instruction	RegDst	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Finalizing Control

- Thus, we can create truth table for each of outputs based on binary encoding of opcodes
- Figure shows logic in control unit as truth table that combines all outputs and that uses opcode bits as inputs
- It completely specifies control function, and we can implement it directly in gates in an automated fashion

Control Function for Single-cycle Implementation

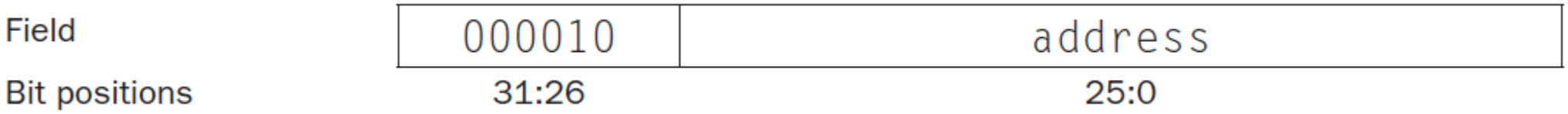
Input or output	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Jump Instruction Implementation

- Let's add jump instruction to show how basic datapath and control can be extended to handle other instructions in the instruction set

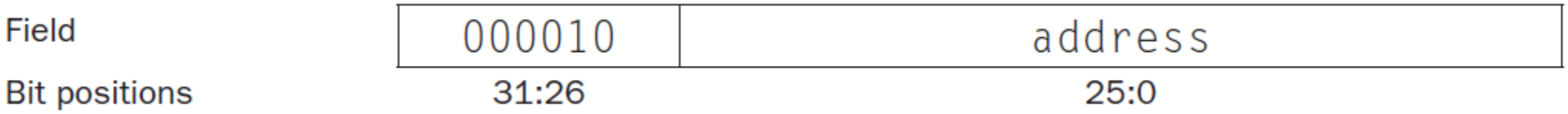
Jump Instruction Implementation

- Jump instruction looks somewhat like a branch instruction but computes target PC differently and is not conditional
  - Like branch, low-order 2 bits of jump address are always 00_{two}
  - Lower 26 bits of this 32-bit address come from the 26-bit immediate field in instruction
  - Upper 4 bits of address that should replace PC come from PC of jump instruction plus 4



Jump Instruction Implementation

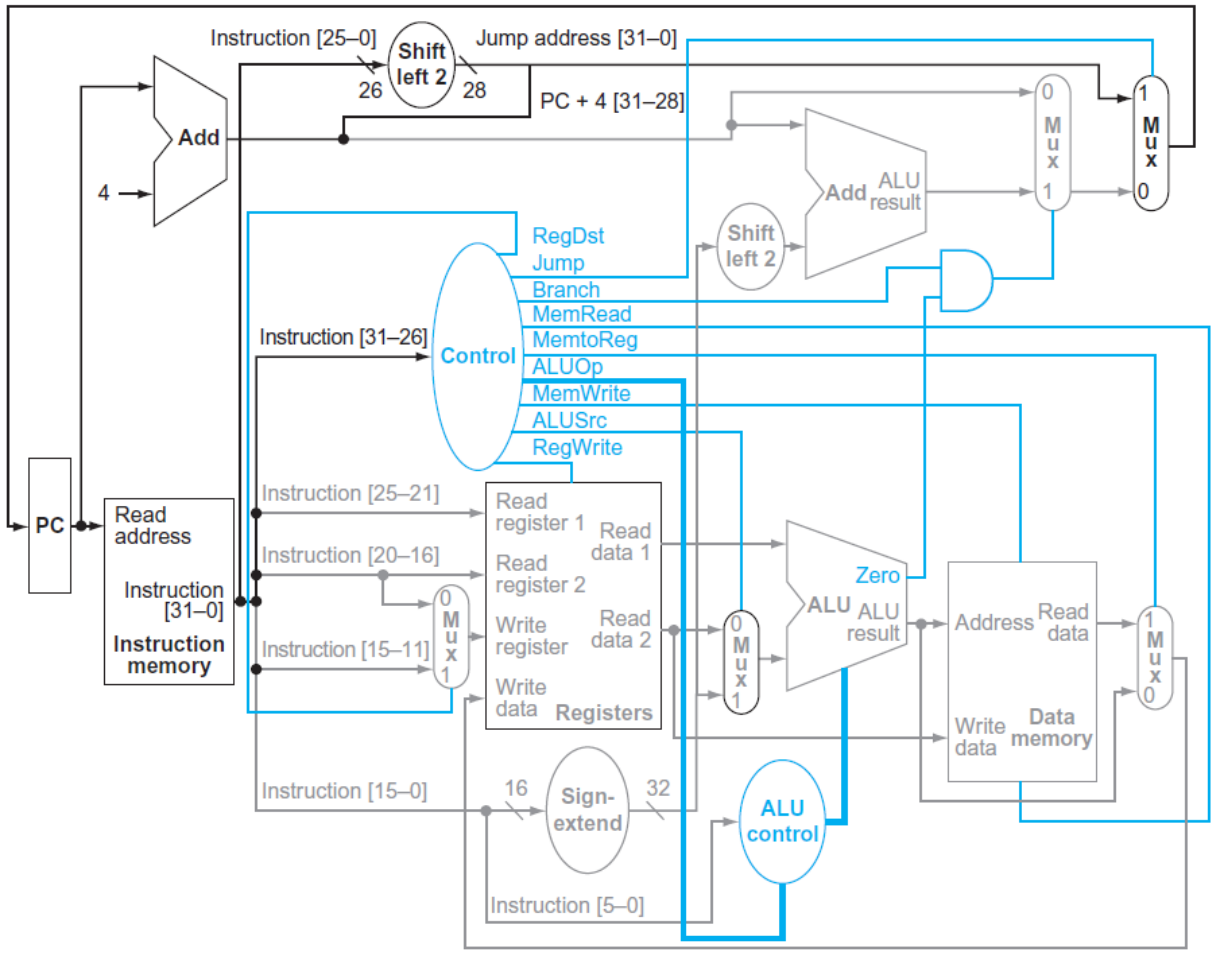
- We can implement jump by storing into PC concatenation of
  - Upper 4 bits of the current PC + 4 (bits 31:28 of the sequentially following instruction address)
  - 26-bit immediate field of the jump instruction
  - Bits 00_{two}



Jump Instruction Implementation

- Figure shows addition of control for jump
- An additional multiplexor is used to select source for new PC value, which is either the incremented PC (PC + 4), branch target PC, or jump target PC
- One additional control signal is needed for the additional multiplexor
  - This control signal, called *Jump*, is asserted only when instruction is a jump; that is, when opcode is 2

Jump Instruction Implementation



Single-cycle Implementation

- Now that we have a single-cycle implementation of most of MIPS core instruction set
- Single-cycle implementation also called single clock cycle implementation
  - An implementation in which an instruction is executed in one clock cycle
  - While easy to understand, it is too slow to be practical

Single-cycle Performance

- Although single-cycle design will work correctly, it would not be used in modern designs because it is inefficient
- To see why this is so, notice that clock cycle must have same length for every instruction in this single-cycle design
  - The longest possible path in processor determines clock cycle
- This path is almost certainly a load instruction, which uses five functional units in series
  - Instruction memory, register file, ALU, data memory, and register file

Single-cycle Performance

- Although CPI is 1, the overall performance of single-cycle implementation is likely to be poor, since clock cycle is too long
- Penalty for using single-cycle design with a fixed clock cycle is significant, but might be considered acceptable for this small instruction set

Single-cycle Performance

- Historically, early computers with very simple instruction sets did use this implementation technique
- However, if we tried to implement the floating-point unit or an instruction set with more complex instructions, this single-cycle design wouldn’t work well at all

Single-cycle Performance

- Because we must assume that clock cycle is equal to the worst-case delay for all instructions, it's useless to try implementation techniques that reduce delay of the common case but do not improve worst-case cycle time
- A single-cycle implementation thus violates the great idea of making the common case fast

Improved Implementation Technique

- Next, we'll look at another implementation technique, called pipelining, that uses a datapath very similar to single-cycle datapath but is much more efficient by having a much higher throughput
- Pipelining improves efficiency by executing multiple instructions simultaneously